Test case: AL-38

Story: As an underwriter, I download a PDF/CSV audit pack for a decision

Tested by: Nikoloz Chilachava
Date run: 10 Nov 2026
Build: `feature/AL-160-audit-pack-download` at commit `7328a15` (PR #192, open; review fix `d040e0b` pushed on 12 Nov)

Preconditions

- Firebase emulators started with the seed data from main's al-core (`firebase emulators:start --project demo-al --import=./seed-data` in `al-core`), and the portal running (`npm run dev` in `al-web`). Seeded logins use the demo password from `al-core/scripts/seed.js`.
- The al-core API running locally against the emulators.
- A local Hardhat node running, with the contracts deployed (`npm run deploy:local` in `al-core/chain`).
- `app-approved` minted on that chain (the AL-37 test case, step 2).
- Signed in as uma@test.com (underwriter).

Steps

#	Criterion	Steps	Expected result	Automated check	Result
1	1	Open the loan file for `app-approved` and click "Download PDF". Open the file.	A PDF with the application summary, valuation, affordability figures, each agent's vote, the credit memo, the underwriter's decision and justification, and the on-chain transaction hashes.	AuditPackActions.test.tsx, auditExports.test.ts	[ ] Pass [ ] Fail
2	2	Click "Download CSV" and open the file in Excel.	The same figures, one field per row, opening cleanly in Excel.	auditExports.test.ts	[ ] Pass [ ] Fail

3	3	Read the hash rows of the CSV and the verification line on the page.	The pack holds the SHA-256 audit hash, and recomputing it from the canonical record gives the hash stored on-chain.	AuditPackActions.test.tsx	[x] Pass [ ] Fail
4	4	Download the pack as the broker of the client (ciara@test.com), then try as another broker.	Underwriters and the client's own broker can download it; anyone else is refused.	AuditPackActions.test.tsx	[ ] Pass [ ] Fail

Automated test run

al-web Vitest on `feature/AL-160-audit-pack-download` at `7328a15`, 10 Nov 2026: **578 passed, 0 failed**. After the review fix `d040e0b`, 12 Nov 2026: **588 passed, 0 failed**. The build (`npm run build`: `tsc -b` then `vite build`) passed.

Notes

- Step 1: "Download PDF" produced `audit-pack-app-approved.pdf`, 13,013 bytes, starting `%PDF-1.4`. Not run yet: opening the PDF and checking each required section against the criterion.
- Step 2: "Download CSV" produced `audit-pack-app-approved.csv`, 6,679 bytes, with rows `section,field,value` and `loan_type heloc` in the application section. Not run yet: opening it in Excel. Quoting of commas, quotes and line breaks is unit tests only (auditExports.test.ts).
- Step 3: `computed_hash`, `stored_hash`, `loan_note_hash` and `on_chain_hash` were all `e68205a7...5065`, with `on_chain_checked` true and `verified` true. The page showed "Verified: the record, the loan note and the registry agree." The hash is recomputed by al-core from the canonical record; the portal shows the result.
- Step 4: Not run yet: the broker's own download and another broker's refusal. Who may fetch the pack is decided by al-core's audit pack endpoint.